基础知识

输入与输出

第一个程序

```
print("Hello world!")
```

- print()用来输出信息
- 字符串：用一对引号包围起来的字符序列。一对引号，可以是一对单引号，也可以是一对双引号，但不可混用。

欢迎程序

```
name = input("What's your name? ")
print("Hello",name)
```

- input() 首先给出提示信息，然后等待用户输入。输入的信息为字符串类型，保存在赋值号前的变量中。
- print()函数会在输出的多个参数之间自动添加一个空格。例如：

```
print("Hello","world")
Hello world
```

- 对比：二个字符串可以用+连接在一起，此时不会在中间自动添加一个空格。如"hello"+"world" => "helloworld"

例子：求圆的周长和面积

- 输入圆的半径，计算圆的周长和面积并输出。
- 输入：圆的半径 radius
- 输出：圆的周长 perimeter 和面积 area
- 提示：圆周率使用math.pi。圆的周长 $perimeter = 2 \times \pi \times radius$，圆的面积 $area = \pi \times radius^2$
- 输入输出示例：
- 输入：

```
5
```

- 输出：

```
31.41592653589793
78.53981633974483
```

```
import math
radius = float(input())
perimeter = 2 * radius * math.pi
area = radius**2 * math.pi
print(perimeter)
print(area)
```

- input() 得到的输入总是字符串，如果输入了数字并计划后续参与运算，应该转换为数字类型。int()可以转换整数，float()可以转换浮点数，而eval()可以计算整个字符串表达式。

例子：计算体质指数BMI

- 体质指数（body mass index，简称BMI）是评价18岁以上成人群体营养状况的指标，可由 体重（单位：千克）和 身高 （单位：米）计算出来。
- BMI的计算公式为：$BMI = 体重 \div 身高^2$
- 输入：体重 weight（单位：千克）和身高 height（单位：米）
- 输出：计算并输出 BMI 值。
- 输入输出示例：
- 输入：

```
60
1.6
```

- 输出：

```
23.437499999999996
```

```
weight = float(input())
height = float(input())
bmi = weight / (height * height)
print(bmi)
```

IPO

- 输入 Input - 处理 Process - 输出 Output
- 建议将程序分成三段，输入、处理、输出，这样条理清晰。

输出语句print()函数

- 可以输出一个或者多个对象，当输出多个对象时，对象之间要用逗号分隔。
- 默认情况下，多个输出的对象之间，用一个空格分隔。可以用参数 sep 设置输出时对象之间的分隔符。

```
print('hello','world','!')
# hello world !
print('hello','world','!',sep='*')
# hello*world*!
print('hello','world','!',sep='')
# helloworld!
```

- 默认情况下，每次输出以换行符结尾。可以用参数 end 设置输出以何字符结尾。
- 参数 end 与 sep 的出现顺序没有要求，但是必须放在要显示的内容之后。
- 单独一个print()没有任何参数，会输出一个空换行。

```
print('hello',end=' ')
print('world',end='')
print(' ',end='!')
print()
print('New line')

# hello world !
# New line
```

变量与赋值

变量

- 变量是计算机语言中能储存计算结果或能表示值的抽象概念。变量可以通过变量名访问。

创建变量

- 赋值语句：`变量名 = 变量值`
- = 叫做赋值运算符。**注意，它并不是数学中的等号！**
- 变量名：可以理解为是门牌号码（引用而不是存储）
- 例如:

```
name = "Larry”
y = 1
radius = 1.0
x = 5 * (3 / 2) + 3 * 2
x = y + 1
area = radius * radius * 3.14159
```

赋值的等号不是数学的等号

- 请不要把赋值语句的等号等同于数学的等号。比如下面的代码：

```
x = 10
x = x + 2
```

- 如果从数学上理解 `x = x + 2` 那无论如何是不成立的。在程序中，处理过程是：

    - 赋值语句先计算右侧的表达式 x+2，得到结果 12
    - 再将结果 12 赋给变量 x
    - 由于 x 之前的值是 10，重新赋值后，x的值变成 12。
- 而且，赋值号的左边只能是变量。比如下面的代码是错误的：

```
1 = x
2 * x = 3 + y
```

连续赋值

```
i = j = k = 1
```

等价于：

```
k = 1
j = k
i = j
```

同时赋值

```
x, y = 2, 3
x, y = x+y, x*y
```

交换值

```
x = 2
y = 3
t = x
x = y
y = t
```

由于Python支持同时赋值，上述过程可以简化为：

```
x, y = y, x
```

变量命名

- 标识符：作为变量、函数等名字的有效字符串集合
- 关键字（keyword）：Python语言保留使用的标识符，又称保留字，不能用于其它用途（如变量名）。使用下面的方法可以列出关键字。

```
import keyword
print(keyword.kwlist)
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class',
'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global',
'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return',
'try', 'while', 'with', 'yield']
```

- 变量名只能含有数字, 字母以及下划线
- 变量名不能以数字开头
- 选择具有描述性的名字，做到望名识义！例如"score"而不是"s"。
- 风格保持一致，常见的变量命名风格有：
    - high_score（下划线命名法）
    - highScore（小驼峰命名法）
    - HighScore（大驼峰命名法）
- 遵循语言习惯
    - Python推荐下划线命名法，也有一些人习惯使用小驼峰命名法（可能是受 Java 语言影响）。这两种风格都可以，但是建议不要在一个程序中混用。
    - 大驼峰命名法一般不得用于变量命名（大驼峰命名法另有用途，如类名或为异常命名）
    - 一般不得使用全大写字母做变量名（如：NUMBER_OF_THE_RESULT），这种命名一般代表常量（即值不会发生变化的变量，虽然 Python 并没有语法对此加以限制）
    - 不用下划线开头或结尾（下划线开头或结尾的变量往往有特殊的用途）
    - 变量名不要太长，最好在15个字符以内
    - 除特殊情况外（如循环控制变量习惯用i），一般不得使用单字母作为变量名
    - 一般使用英文单词命名，汉语环境中很难对应翻译为英文可以用拼音，但是不允许英拼混用（错误示例：RunYear代表闰年），更不允许使用拼音首字母缩写（错误示例：ncwssmys你猜我是什么意思）
    - 不允许在变量名中出现汉字，尽管Python看起来也能正确处理。

数据类型、运算符与表达式

数据类型

- 计算机处理的数据，是要区分数据类型的。
- Python 中数据类型，主要分为基本数据类型、内置容器类型和自定义类型
    - 基本数据类型：整数int(integer)、浮点数float、复数complex、布尔型bool(boolean)等
    - 内置容器类型：字符串型str(string)、元组tuple、列表list、字典dict(dictionary)、集合set等
    - 自定义类型：自定义类、第三方类等
- 本章主要介绍整数和浮点数类型，其它类型的数据，将在后续章节中展开介绍。

运算符

- 运算符用于执行程序代码运算，会针对一个及以上操作数项目来进行运算。
- 按操作数个数分，可分为一元运算符、二元运算符等。
- 大部分运算符为二元运算符，如a+b，+即为加法运算符，它的操作数有二个：a和b 。一般来说，二元运算符位于二个操作数之间。
- 部分运算符为一元运算符，如-a，-为取负运算符，它只有a这一个操作数，意为负的a，如果a为5，则-a为-5，而如果a为-3，则-a为3。一般来说，一元运算符位于操作数前面。
- 按作用来分，Python 中的运算符包括算术运算符、比较运算符、逻辑运算符、赋值运算符、位运算符、成员运算符和身份运算符等。
- 本章主要介绍用于数字型数据的算术运算和比较运算，也会介绍逻辑运算、赋值运算的使用，部分其它类型的运算，将在后续章节中展开介绍。

表达式

- 表达式一般由运算符和操作数（操作对象）组成。例如，1+2，+为二元运算符，1和2是操作数。
- 仅操作数也可以构成表达式。如：1

数字类型和运算

数字型

- 整数：没有小数部分的数字，例如：1, 27, -100, 0 . Python中的整数即数学中的整数，理论上 Python 可以处理任意大小的整数，包括正整数、零和负整数。
- 浮点数：带有小数点的数字，例如：2.376, -99.1, 1.0 . Python中的浮点数并不等价于数学中的小数，Python 并不能处理所有的小数。
- 复数：形如 a+bj 的数字，例如：2+3j

关于进制

- 人类社会习惯使用十进制，大概是因为人一般有10根手指，在远古运算时天然形成“逢十进一”的习惯。
- 计算机中大量使用的光、电、磁，都只有二种状态（光或电流的有无、正负电荷、南北磁极），所以计算机天然形成了“逢二进一”的习惯，即二进制。
- 用二进制表示的数字太长，人类阅读不便，从而产生了八进制和十六进制。其中十六进制需要借用"A~F"代表10～15。

进制	前缀	转换函数	示例
十进制	无	int()	int(0x61) -> 97
二进制	0b	bin()	bin(97) -> 0b1100001
八进制	0o	oct()	oct(97) -> 0o141
十六进制	0x	hex()	hex(97) -> 0x61

关于浮点数的说明

- 浮点数对应数学上的小数，但是 Python 并不能处理数学上所有的小数。
- 浮点数大小有限制。不能处理绝对值过大或过小的小数。绝对值最大的浮点数为 1.7976931348623157e+308（科学计数法，记为aen或aEn，其中a代表尾数，n代表指数，e或E是固定的字符，作为尾数和指数的分隔，含义为a 乘以10的n次方），绝对值最小的浮点数为 2.2250738585072014e-308。
- 浮点数精度有限制。浮点数只有15位有效数字，超长的部分没有意义。
- 之所以称为浮点数，是因为按照科学记数法表示时，一个浮点数的小数点位置是可变的，比如，1.23乘以10的9次方 和 12.3乘以10的8次方 是完全相等的。浮点数可以用数学写法，如1.23，3.14，-9.01，等等。但是对于很大或很小的浮点数，就必须用科学计数法表示，把10用e替代，`1.23*10**9`就是1.23e9，或者12.3e8，0.000012可以写成1.2e-5，等等。
- 整数和浮点数在计算机内部存储的方式是不同的，整数运算永远是精确的，而浮点数运算则可能会有误差。

  > 著名的 0.1+0.2不等于0.3问题
  >
  > 计算机能正确的计算整数，如1+2=3，但是，如果用计算机计算0.1+0.2，却得不到0.3！
  >
  > > 0.1 + 0.2 = 0.30000000000000004
  >
  > 这并不是 Python 出错了，而是大多数计算机程序都会得到的结果。为此专门还有人建立了一个网站，叫 http://0.30000000000000004.com (3和4之间有15个0)

- 同样的，二个浮点数比较是否相等，是很容易出问题的。如下例，如果判断a是否等于b，那结果是不等于，但是如果判断a减b的绝对值是否小于 eps，如果小于eps，就可以近似的认为a等于b 。其中eps的值由实际应用需要而定。

```
a = 0.1+0.2
b = 0.3
eps = 0.000001
print(a==b)          # a等于b吗？False
print(abs(a-b)<eps) # a近似等于b吗？True
```

- 解决这类问题的方法，一种方法是尽量使用整数，如处理与金钱相关的数值时，以分为单位。另一种方法是使用decimal模块。

```
from decimal import Decimal
a = Decimal('0.1')
b = Decimal('0.2')
print(a+b) # 会输出 0.3
```

算术运算

算术运算符

- 用于数字的算术运算符，与数学中的运算基本一致，只有除法分成了三种运算符。
- Python 不认识×号，所以乘法运算用*代替。
- Python 不认识÷号，所以除法运算用/代替。

运算符	描述	举例
+	一元运算符，一元加，+a 即得到该操作数本身不变	如a=-10，则+a仍然是-10
-	一元运算符，一元减，-a 即得到变换该操作数的符号的数	如a=-10，则-a是10
+	二元运算符，加法，a+b 两个对象相加	10 + 20 结果 30
-	二元运算符，减法，a-b 两个对象相减	10 - 20 结果 -10
*	二元运算符，乘法，a*b 两个对象相乘。	10 * 20 结果 200
/	二元运算符，算术除法（truediv），a/b 结果总是浮点数。	15 / 2 结果 7.5
//	二元运算符，地板除法（floordiv），a//b 返回除法的商。	10 // 3 结果是 3
%	二元运算符，取模运算（mod），又叫取余运算，a%b 返回除法的余数。	10 % 3 结果是 1
**	二元运算符，乘方运算，又叫幂运算，a**b 返回a的b次幂	2 ** 3 结果是 8

算术运算符的优先级

- 与数学中的先乘除、后加减的常识相同，用于数字的算术运算符的优先级为（从高到低）：
    - 幂运算符
    - 一元运算符：一元加、一元减
    - 乘除运算符：乘法、算术除法、地板除法、取模（取余）
    - 加减运算符：二元加、二元减
- 同样的，可以使用小括号来调整运算的优先级。注意与数学中不同，调整运算优先级时永远使用小括号，无论有多少级。中括号和大括号在Python中另有用途。

关于除法的说明

- 算术除法（truediv），算术除法会得到较精确的结果，结果总是浮点数，即使结果应为整数，也要在其后加上.0转为浮点数的形式。如：

```
print(5/2) # 2.5
print(6/2) # 3.0
print(10/3) # 3.3333333333333335
```

- 地板数：指不大于此数的最大整数。如：

```
import math
print(math.floor(-3.5)) # -4
print(math.floor(-3))   # -3
print(math.floor(3.5))  # 3
print(math.floor(3))    # 3
```

- 地板除法（floor），地板除法得到算术除法结果的地板数，即除法的商。如果两个操作数中有浮点数，则结果以浮点数形式表示，即加上.0

```
print(10//3)    # 3
print(9//3)     # 3
print(-9//3)    # -3
print(-10//3)   # -4
print(10//3.5)  # 2.0
print(-10//3.5) # -3.0
```

- 取余（取模）运算，得到除法的余数。具体的公式如下：设 x // y = z，则 x % y = x – (y*z)

```
print(10%3)     # 1
print(-10%3)    # 2
print(-10%-3)   # -1
print(-10%-3.5) # -3.0
```

例子：求n天后是周几

- 如果已知今天是周四，求n天后是周几?
- 用1代表周一，2代表周二，.....，6代表周六，0代表周日

```
n = int(input())
print( ( 4 + n ) % 7 )
```

例子：反转输出一个四位整数

- 输入：一个四位（1000～9999之间）整数
- 输出：反转各位数字后显示。
- 提示：反转后如果最高位为0的，不需要显示最高位的0。

示例	输入	输出
1	3125	5213
2	8670	768

```
n = eval(input())
a = n // 1000        # 千位
b = n // 100 % 10    # 百位
c = n // 10 % 10     # 十位
d = n % 10           # 个位
m = d * 1000 + c * 100 + b * 10 + a
print(m)
```

常用内置数学函数

函数	描述	举例
abs(x)	返回x的绝对值	abs(-2)=2
max(x1,x2,...)	返回x1,x2,...的最大值	max(1,5,2)=5
min(x1,x2,...)	返回x1,x2,...的最小值	min(1,5,2)=1
pow(a,b)	返回a的b次方值，等价于a**b	pow(2,3)=8
round(x)	返回与x最接近的整数。如果x与两个整数接近程度相同，则返回偶数值	round(4.5)=4
round(x,n)	保留浮点数点后n位小数的浮点数值	round(5.466,2)=5.47

- round(x)函数与四舍五入的比较。一般情况下round(x)相当于数学上的将x四舍五入到整数，但是当x的小数部分恰好为.5时，即x与上下两个整数接近程度相同，则返回两个整数中的偶数值。如round(3.5)=4; round(4.5)=4; round(5.5)=6; round(-2.5)=-2; round(-3.5)=-4
- 对比：int(x)是将x的小数部分直接截断丢弃。如 int(4.5)=4; int(-3.5)=-3
- 当x大于等于0时，可以使用 int(x+0.5) 得到数学上的将x四舍五入到整数的结果。如 int(3.5+0.5)=4; int(4.5+0.5)=5.
- 当x小于0时，可以使用 int(x-0.5) 得到数学上的将x四舍五入到整数的结果。如：int(-3.5-0.5)=-4; int(-4.5-0.5)=5.
- 上述二种情况，可以合并成一个表达式：int(x+0.5) if x>=0 else int(x-0.5)
- 使用round(x,n)时，如果n>0，代表保留小数点后n位数字，如果n<0，则代表向小数点前保留-n位数字，默认n=0为保留到整数。如：round(1234.5678,2)=1234.57; round(1234.5678,-2)=1200.0
- 由于浮点数的不精确性，当使用round(x,n)函数的参数n>0时，遇到被保留位后一位是5并且之后没有更多数字时，所得结果往往不可预测。这个问题与0.1+0.2!=0.3一样，都是浮点数在计算机中以二进制小数保存有关。如：round(5.215,2)=5.21; round(5.225,2)=5.22 ，还有：round(1.235,2)=1.24; round(1.335,2)=1.33

常用math库数学函数

函数/常量	描述	举例
math.pi	圆周率PI常数	math.pi=3.141592653589793
math.e	自然对数的底数e常数	math.e=2.718281828459045
math.fabs(x)	将x看作一个浮点数，返回它的绝对值	math.fabs(-2)=2.0
math.ceil(x)	x向上取最近的整数，即不小于x的最小整数	math.ceil(2.1)=3; math.ceil(-2.1)=-2
math.floor(x)	x向下取最近的整数，即不大于x的最大整数	math.floor(2.1)=2; math.floor(-2.1)=-3
math.exp(x)	常数e的x次方值	math.exp(1)=2.718281828459045
math.log(x)	x的自然对数，即x以常数e为底数的对数值	math.log(math.e)=1.0

函数/常量	描述	举例
math.log(x,base)	x以base为底数的对数值	math.log(100,10)=2.0
math.sqrt(x)	x的平方根值	math.sqrt(4)=2.0
math.sin(x)	x的正弦函数值，x为弧度值	math.sin(math.pi/2)=1.0
math.asin(x)	x的反正弦函数值，结果为弧度值	math.asin(1.0)=1.5707963267948966
math.cos(x)	x的余弦函数值，x为弧度值	math.cos(math.pi)=-1.0
math.acos(x)	x的反余弦函数值，结果为弧度值	math.acos(1.0)=0.0
math.tan(x)	x的正切函数值，x为弧度值	math.tan(0.0)=0.0
math.atan(x)	x的反正弦函数值，结果为弧度值	math.atan(0.0)=0.0
math.degrees(x)	将弧度值转换为角度值	math.degrees(math.pi)=180.0
math.radians(x)	将角度值转换为弧度值	math.radians(90)=1.5707963267948966

数字类型转换

整数与浮点数相互类型转换

- 浮点数转整数：int(x)，当x为浮点数时，将x的小数部分直接截断丢弃转为整数。如：int(4.5)=4; int(-3.5)=-3
- 整数转浮点数：float(x)，当x为整数时，在x的后面补上.0转为浮点数。如：float(4)=4.0

浮点数转换为整数的几个函数的比较

- int(x)：当x为浮点数时，将x的小数部分直接截断丢弃转为整数。
- math.floor(x)：x向下取最近的整数，即不大于x的最大整数。
- math.ceil(x)：x向上取最近的整数，即不小于x的最小整数。
- round(x)：返回与x最接近的整数。如果x与两个整数接近程度相同，则返回偶数值。
- int(x+0.5) if x>=0 else int(x-0.5)：返回数学上将x四舍五入到整数的值。

比较运算

- 比较运算的结果为布尔值，又叫逻辑值，只有两个值：True 和 False
- 注意有二个符号的运算符是一个整体，其中的二个符号中间不可插入空格
- 注意 `==` 相当于数学上的等于号，而=是赋值号，与 `==` 完全不同。

运算符	描述	举例
==	等于，a==b，当a等于b时为True，否则为False	10==20为False
!=	不等于，a!=b，当a不等于b时为True，否则为False	10!=20为True
<	小于，a<b，当a小于b时为True，否则为False	10<20为True

运算符	描述	举例
<=	小于等于，a<=b，当a小于等于b时为True，否则为False	10<=20为True
>	大于，a>b，当a大于b时为True，否则为False	10>20为False
>=	大于等于，a>=b，当a大于等于b时为True，否则为False	10>=20为False

例子：显示当前时间

- 说明：time.time() 获得现在距离 Unix Epoch（1970-01-01 00:00:00 UTC）所经过的时间，整数部分单位为秒，小数部分代表毫秒。

```
import time

currentTime = time.time() # Get current time

# Obtain the total seconds since midnight, Jan 1, 1970
totalSeconds = int(currentTime)

# Get the current second
currentSecond = totalSeconds % 60

# Obtain the total minutes
totalMinutes = totalSeconds // 60

# Compute the current minute in the hour
currentMinute = totalMinutes % 60

# Obtain the total hours
totalHours = totalMinutes // 60

# Compute the current hour
currentHour = totalHours % 24

# Display results
print("Current time is " + str(currentHour) + ":"
    + str(currentMinute) + ":" + str(currentSecond) + " GMT")
```

例子：房贷计算器

- 输入：贷款年利率、贷款年限、总贷款额
- 输出：月供、总还款额
- 设每月还款额相同。计算公式如下：

```
月利率 = 年利率 / 12
月供 = 总贷款额 * 月利率 / (1 - 1/(1+月利率)**(贷款年限*12) )
总还款额 = 月供 * 贷款年限 * 12
```

```
# Enter yearly interest rate
```

```
annualInterestRate = eval(input(
  "Enter annual interest rate, e.g., 8.25: "))
monthlyInterestRate = annualInterestRate / 1200

# Enter number of years
numberOfYears = eval(input(
  "Enter number of years as an integer, e.g., 5: "))

# Enter loan amount
loanAmount = eval(input("Enter loan amount, e.g., 120000.95: "))

# Calculate payment
monthlyPayment = loanAmount * monthlyInterestRate / (1
  - 1 / (1 + monthlyInterestRate) ** (numberOfYears * 12))
totalPayment = monthlyPayment * numberOfYears * 12

# Display results
print("The monthly payment is " +
  str(int(monthlyPayment * 100) / 100))
print("The total payment is " + str(int(totalPayment * 100) /100))
```

例子：计算二点间距离

```
# Enter the first point with two double values
x1, y1 = eval(input("Enter x1 and y1: "))

# Enter the second point with two double values
x2, y2 = eval(input("Enter x2 and y2: "))

# Compute the distance
distance = ((x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 - y2)) ** 0.5

print("The distance between the two points is", distance)
```

例子：计算三角形内角

- 输入三角形三个顶点的坐标 (x1,y1), (x2,y2), (x3,y3)
- 可以求出三角形三边边长 a,b,c
- 由边长a, b, c，可以求得三个内角A, B, C

```
import math

x1, y1, x2, y2, x3, y3 = eval(input("Enter three points: "))

a = math.sqrt((x2 - x3) * (x2 - x3) + (y2 - y3) * (y2 - y3))
b = math.sqrt((x1 - x3) * (x1 - x3) + (y1 - y3) * (y1 - y3))
c = math.sqrt((x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 - y2))

A = math.degrees(math.acos((a * a - b * b - c * c) / (-2 * b * c)))
```

```
B = math.degrees(math.acos((b * b - a * a - c * c) / (-2 * a * c)))
C = math.degrees(math.acos((c * c - b * b - a * a) / (-2 * a * b)))

print("The three angles are ", round(A * 100) / 100.0,
    round(B * 100) / 100.0, round(C * 100) / 100.0)
```

字符类型

- 计算机只认识0和1，不认识人类语言中的字符，所以要把字符进行编码后，计算机才可以使用字符。
- Python 没有独立的字符类型，它并不区分字符和字符串，即使只是一个字符，也被当做字符串处理。

字符编码

- 最早的时候，计算机只能处理英文字符、数字和常用标点。编码字符集叫ASCII。ASCII编码共收录了 128 个字符，包含了基本的英文字母（包括大小写）、数字（也就是 1234567890）、标点符号（,.!等）、特殊符号（@#$%^&等）以及一些具有控制功能的字符（往往不会显示出来）。
- Python3 缺省使用 UTF-8 编码。ASCII编码中的包含的字符，在 ASCII 与 UTF-8 中编码保持一致。UTF-8 编码使得 Python 很容易处理包括中文在内的多国语言文字。

常用ASCII码

编码	字符		编码	字符		编码	字符
48	0	|	65	A	|	97	a
49	1	|	66	B	|	98	b
50	2	|	67	C	|	99	c
.....	.....	|	.....	.....	|	.....	.....
55	7	|	88	X	|	120	x
56	8	|	89	Y	|	121	y
57	9	|	90	Z	|	122	z

字符与编码的转换

- 使用ord(char)可以得到char字符的编码。
- 使用chr(number)可以得到编码为number的字符。

```
>>> ord('a')
97
>>> ord('中')
20013
>>> chr(97)
'a'
>>> chr(22269)
'国'
```

- 字符比较大小的依据是它的编码，编码越大，字符在比较时就越大。

```
>>> ord('A')
65
>>> ord('a')
97
>>> 'A'<'a'
True
```

区分数字与字符

- 计算机中数字类型与字符类型是严格区别的二种数据类型。其中容易给人造成困扰的字符中的数字字符。
- 数字类型是可以进行数学运算的，比如 `1+2=3` ，这里面的 1、2、3 都是数字，可以进行数学运算。
- 字符中的数字字符，如 '1'、'2'、'3'，本质上也是字符，与 'a'、'b'、'c' 一样，只能用来组成字符串，不能进行数学运算。比如 `'1'+'2'='12'` ，符合字符串相加代表连接的运算规则。

数字与字符串类型转换

字符串转数字

- int(x)：可将形为整数的字符串x转为整数，如int("45")=45. 注意x不可为形为浮点数的字符串，如：int("4.5")是会出错误的。
- float(x)：可将形为浮点数或整数的字符串x转为浮点数，如：float("45")=45.0; float("45.6")=45.6
- eval(x)：可以将字符串x视为一个表达式，计算其值。如：eval("1+2")=3

数字转字符串

- str(x)：可将整数或浮点数 x 转为字符串。当需要将一个字符串与数字相连接在一起时就需要使用此函数。如：

```
a=2
b=3
s="The result of "+str(a)+"+"+str(b)+" is "+str(a+b)
print(s)
```

逻辑（布尔）运算

逻辑（布尔）值

- 二值逻辑：True、False

其它类型的数值参与逻辑运算

- 数字 0、0.0、0j 被视同 False，其它数字被视同 True
- 空字符串、空元组、空列表、空字典、空集合，视同 False，上述数据类型的非空值视同 True
- None被视同False
- 可以使用 bool() 将其它数据类型转换为布尔类型值。

```
print(bool(0))  # False
print(bool(1))  # True
print(bool('')) # False
print(bool('False'))    # True
```

逻辑（布尔）运算符

- 三种运算：逻辑非 not、逻辑与 and、逻辑或 or
- 运算优先级：not > and > or

逻辑非 not

- 一元运算符，表示“相反”。not True即为False，not False即为True。
- 逻辑非的真值表如下：

操作数A	not A
True	False
False	True

- 逻辑非运算的返回结果：只有True和False二值

```
print(not 1)    # False
print(not '')   # True
```

逻辑与 and

- 二元运算符，表示“并且”，要求二个操作数均为 True 时，结果为 True，其它情况结果均为 False。
- 逻辑与的真值表如下：

操作数A	操作数B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

- 短路运算规则：如果已经判定第一个操作数是 False，则不再判断第二个操作数。
- 逻辑与运算的返回结果：最早确定表达式值的操作数。即，表达式 a and b 的返回值，如果 a 是 False，则返回 a，如果 a 是 True，则返回 b。

```
print(6 and 'lucky')    # 'lucky'
print(0 and '')         # 0
```

逻辑或 or

- 二元运算符，表示"或者"，要求二个操作数均为 False 时，结果为 False，其它情况结果均为 True。
- 逻辑或的真值表如下：

操作数A	操作数B	A or B
True	True	True
True	False	True
False	True	True
False	False	False

- 短路运算规则：如果已经判定第一个操作数是 True，则不再判断第二个操作数。
- 逻辑或运算的返回结果：最早确定表达式值的操作数。即，表达式 a or b 的返回值，如果 a 是 True，则返回 a，如果 a 是 False，则返回b。

```
print(6 or 'lucky') # 6
print(0 or '')      # ''
```

复合赋值运算

- 注意复合赋值运算符是一个整体，其中的二个符号中间不可插入空格
- 下面示例中设初始值 a=10

运算符	描述	举例
+=	a+=b，即a=a+b	a+=1，结果a的值变为11
-=	a-=b，即a=a-b	a-=1，结果a的值变为9
*=	a*=b，即a=a*b	a*=3，结果a的值变为30
/=	a/=b，即a=a/b	a/=3，结果a的值变为3.3333333333333335
//=	a//=b，即a=a//b	a//=3，结果a的值变为3
%=	a%=b，即a=a%b	a%=3，结果a的值变为1
**=	a**=b，即a=a**b	a**=2，结果a的值变为100

运算符优先级

- 下表描述了常用运算符的优先级，从上向下优先级为由高到低。
- 必要时，尽量使用小括号使代码逻辑更清晰。

运算符	描述	操作数个数
**	幂运算符	二元

运算符	描述	操作数个数
+ -	一元运算符：一元加、一元减	一元
* / // %	乘除运算符：乘法、算术除法、地板除法、取模（取余）	二元
+ -	加减运算符：二元加、二元减	二元
<= < > >= == !=	比较运算符	二元
not	逻辑非	一元
and	逻辑与	二元
or	逻辑或	二元
= += -= *= /= //= %= **=	赋值运算符	二元

注释

注释的作用

- 注释（Comments）用来向用户提示或解释某些代码的作用和功能，它可以出现在代码中的任何位置。解释器在执行代码时会忽略注释，不做任何处理，就好像它不存在一样。
- 在调试（Debug）程序的过程中，注释还可以用来临时移除无用的代码。如果觉得某段代码可能有问题，可以先把这段代码注释起来，让 Python 解释器忽略这段代码，然后再运行调试。
- 注释的最大作用是提高程序的可读性，没有注释的程序简直就是天书。
- 虽然良好的代码可以自成文挡，为了让今后阅读这段代码的人，能够理解你的思路。或者一段时间以后，你自己也不清楚当时写这段代码的目的了。所以程序应该书写适量的注释。
- 一般情况下，合理的代码注释应该占源代码的 1/3 左右。

单行注释

- Python 使用 # 号实现单行注释。# 注释可以在一行中从头开始，也可以从中间开始。
- 从 # 开始，直到这行结束为止的所有内容都是注释。Python 解释器遇到 `#` 时，会忽略它后面的整行内容。

```
#使用print输出字符串
print("Hello World!")
print("Python is fun.")

#使用 print输出数字
print(100)
print( 3 + 100 * 2)
print( (3 + 100) * 2 )

print("Hello World!")    #输出Python欢迎信息
print( 36.7 * 14.5 )     #输出乘积
print( 100 % 7 )         #输出余数
```

多行注释

- 多行注释指的是一次性注释程序中多行的内容（包含一行）。
- Python 使用三个连续的单引号'''或者三个连续的双引号"""注释多行内容。
- 多行注释通常用来为 Python 文件、模块、类或者函数等添加版权或者功能描述信息。

```
'''
使用 3 个单引号分别作为注释的开头和结尾
可以一次性注释多行内容
这里面的内容全部是注释内容
'''

"""
使用 3 个双引号分别作为注释的开头和结尾
可以一次性注释多行内容
这里面的内容全部是注释内容
"""
```

- Python 多行注释不支持嵌套，所以下面的写法是错误的：

```
'''
外层注释
    '''
    内层注释
    '''
'''
```

- 不管是多行注释还是单行注释，当注释符作为字符串的一部分出现时，就不能再将它们视为注释标记，而应该看做正常代码的一部分。

```
print('''Hello,World!''')
print("""Python is fun.""")
print("#是单行注释的开始")
```